

# Claude Code Bootstrap Prompt — Frigoglass RPD Platform

Paste everything below the line into a fresh Claude Code session, opened in an empty repo. Before you do, copy the two reference files into [reference/](#):

```
mkdir -p reference
```

```
cp RPD_Web_Application_mock-up-Frigoglass.pptx reference/
```

```
cp rpd-platform-prototype.html reference/
```

---

You are bootstrapping a multi-agent development environment for a production enterprise application. **In this session you will write ZERO application code.** Your only job is to create the governance scaffolding specified below, exactly as specified. Application code begins in the next session, driven by the agents you are about to create.

Work through this top to bottom. Create every file. Do not summarise, do not abbreviate, do not substitute your own judgement for the specifications given here.

---

## 1. Project context

We are building the **RPD Web Application** for **Frigoglass**, a commercial refrigeration manufacturer (HQ Athens; R&D and production hubs in Greece, India and Romania). It replaces a spreadsheet-based process for planning their annual R&D and Product Development portfolio.

It is delivered to the client and **hosted on Frigoglass's own servers**, on-premise, behind their corporate network. It is not a SaaS product.

The application manages ~236 active projects across 6 hubs, and its central function is a **resource-constrained scheduling engine** that assigns 14 workflow steps per project to engineers and lab chambers across a 78-week horizon, then reports which projects will complete inside the calendar year.

Reference material (read both before writing anything)

| Path                                                               | What it is                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>reference/RPD_Web_Application_mock-up-Frigoglass.pptx</code> | Client's requirement deck. Slide 3 is the scheduling logic flowchart. Slide 4 is the prioritization matrix. Slides 5–11 are UI mockups of the six surfaces.                                                                                                                                                                                                                                                                                                                                                                |
| <code>reference/rpd-platform-prototype.html</code>                 | A working single-file React prototype built by a teammate, using synthetic demo data. It contains a complete, runnable implementation of the domain model and scheduler. <b>It is neither the specification nor the architecture.</b> It is a second opinion — useful as a differential test oracle during P2 and as a source of domain constants to cross-check against the deck. It contains known defects (see DOMAIN_RULES.md). Where our implementation and the prototype disagree, the deck and DOMAIN_RULES.md win. |

To read the deck: `python -c "import zipfile; zipfile.ZipFile('reference/RPD_Web_Application_mock-up-Frigoglass.pptx').extractall('reference/deck_unpacked')"` then inspect `reference/deck_unpacked/ppt/media/*.png` — the slides are largely full-bleed screenshots.

The prototype is a minified esbuild bundle. Search it for the domain constants and the scheduling function; do not attempt to un-minify the whole file.

---

## 2. Locked tech stack

This was decided already. Agents may not substitute alternatives without an entry in `docs/MEMORY.md` recording the decision and its rationale.

### Backend

- Python 3.13, **FastAPI**, Pydantic v2 (strict mode)

- **PostgreSQL 17** — SQLAlchemy 2.0 (async) + Alembic
- **Redis 7** — cache, solver-result memoisation, rate limiting, SSE pub/sub
- **Celery** (Redis broker) — background jobs, solver dispatch, progress, cancel
- **MinIO** — S3-compatible object storage, self-hosted (exports, attachments, large snapshots)
- **OR-Tools CP-SAT** — the optimising scheduler
- Unicorn + Uvicorn workers

## Frontend

- **React 19 + TypeScript + Vite**
- **TanStack Query** (server state) + **Zustand** (UI/scenario state)
- **TanStack Table + TanStack Virtual** — matrix and large grids
- **Tailwind CSS + shadcn/ui**, components sourced from **21st.dev** where they fit
- **Framer Motion** — transitions, layout animation, micro-interactions
- **Lottie** (**lottie-react**) — SCOPED: empty states, solver progress, success confirmations only
- **ECharts** — dense heatmaps and utilization matrices; **Recharts** — simple bar/donut panels
- **Custom Gantt** — hand-built virtualized SVG/Canvas. Do NOT introduce DHTMLX, Bryntum, Syncfusion or any commercial Gantt library.
- **three.js** — NOT USED in v1. Permitted only behind an explicit future feature flag for a 3D cooler-model viewer on Project Registration. It must never appear on Dashboard, Capacity, Matrix, Gantt or Planning. Any agent proposing 3D on those surfaces is wrong.

## Infrastructure & security

- Docker Compose (nginx, api, worker, solver-worker, postgres, redis, minio). **Not Kubernetes.**
- OIDC SSO against the client IdP (assume Microsoft Entra ID; Keycloak as fallback)
- RBAC + row-level hub scoping enforced in the data access layer
- Immutable append-only audit log
- Nginx/Traefik reverse proxy, TLS 1.3, HSTS, ModSecurity + OWASP CRS
- Vault or Docker secrets. Never committed **.env**

---

## 3. Files to create

CLAUDE.md

docs/

PROJECT\_AND\_STACK.md

DOMAIN\_RULES.md

IMPLEMENTATION\_PLAN.md

MEMORY.md

OPEN\_QUESTIONS.md

ADR/

0001-record-architecture-decisions.md

.claude/

agents/

rpd-orchestrator.md

backend-builder.md

algorithm-engineer.md

frontend-builder.md

qa-inspector.md

workflow-auditor.md

security-auditor.md

skills/

ui-ux-pro-max/SKILL.md

frontend-builder/SKILL.md

dataviz-gantt/SKILL.md

scheduling-algorithms/SKILL.md

security-review/SKILL.md

qa-testing/SKILL.md

commands/

start-task.md

close-task.md

gate-check.md

settings.json

---

## 4. **CLAUDE.md** — root context, always loaded

Keep it under 200 lines. It must contain:

1. **One-paragraph project description** (from §1 above).
2. **The Memory Protocol**, stated as a hard rule:

Before starting ANY task, in this order, read:

1. **docs/DOMAIN\_RULES.md** — the business logic contract
2. **docs/MEMORY.md** — what has already been decided, built, and broken
3. **docs/IMPLEMENTATION\_PLAN.md** — the task you are about to do and its dependencies

After completing ANY task, append an entry to **docs/MEMORY.md**. Never edit or delete existing entries — the file is append-only. Never mark a task complete in **IMPLEMENTATION\_PLAN.md** without a corresponding **MEMORY.md** entry.

3. **The Gate Protocol**:

No phase advances until **qa-inspector**, **workflow-auditor** and **security-auditor** have each recorded a PASS for that phase in **docs/MEMORY.md**. The orchestrator enforces this. A FAIL blocks the phase and creates a remediation task at the top of the current phase.

#### 4. The Delegation Rule:

The orchestrator does not write application code. It plans, delegates to subagents, reviews their output against DOMAIN\_RULES.md, and enforces gates.

#### 5. Non-negotiables — a short list:

- The scheduler's behaviour is defined by docs/DOMAIN\_RULES.md, never by the prototype. During P2 the prototype is used as a differential oracle: every divergence between our implementation and its output must be classified as either an intentional fix (ADR required) or a bug in our port. Undocumented divergence fails the gate; documented divergence does not. The oracle is retired when P2 closes.
- Never run CP-SAT inside the FastAPI process.
- No secrets in the repo. No dangerouslySetInnerHTML. No raw SQL string interpolation.
- Financial fields (TCOGS, gross margin, selling price, customer name) are commercially sensitive — encrypted at rest, never logged.
- Engineer-level utilization data is GDPR personal data. See OPEN\_QUESTIONS.md #8.

#### 2. Stack summary table (condensed from §2).

#### 3. Repo layout and the standard commands (make dev, make test, make lint).

---

## 5. docs/PROJECT\_AND\_STACK.md

The full specification. Sections:

1. **Business context** — who Frigoglass is, what problem this solves, who the users are (Portfolio Manager, Hub Planner, Engineer, Executive Viewer, Auditor, Admin).
2. **The six surfaces**, one subsection each, derived from the deck screenshots:
  - **Global RPD Dashboard** — pipeline totals (spillover / newly registered / total), status overview cards, hub × type pipeline summary, completing-within-year table, pipeline-vs-completing chart, analytics breakdown with filters
  - **RPD Capacity** — design & lab capacity vs load per hub, class A+/A/B/C deliverable vs left-out breakdown, load-vs-capacity charts, global resource utilization matrix

- **Prioritization Matrix** — 13-dimension scoring grid, portfolio decision summary, currency toggle (EUR/USD/INR), financial columns split by new-models vs RM-saving projects, scenario simulation, Apply Priorities
  - **Project Execution Timeline** — the Gantt. Week-bucketed, per-project expandable to per-step rows, solid = planned, hatched = actual+delay, red connector = delay, ENG CONFLICT / OVERLAP / LEFT OUT badges, activity-load bottleneck strip, freeze toggle
  - **Project Registration** — create/edit projects, assign leader, category, type, hub, actual start, financials, hard gates
  - **Capacity Planning** — configure engineers (FTE, allowed categories) and chambers (platforms, efficiency, weeks/chamber, allowed stages) per hub; Apply Logic; Auto-assign
  - Plus **Versions & History, Downloads** (CSV/XLSX export of every surface), **Notifications**
3. **Full tech stack** (§2 verbatim) with a one-line justification per choice.
  4. **Architecture** — request flow, where the solver runs, how progress streams to the client over SSE, how scenarios are snapshotted.
  5. **Security model** — the layer table from §2, plus the RBAC role/permission matrix.
  6. **Deployment topology** — the Docker Compose service graph, volumes, backup strategy.
  7. **Non-goals for v1** — multi-tenant, mobile app, 3D visualisation, real-time collaborative editing, ERP integration.

---

## 6. docs/DOMAIN\_RULES.md — THE MOST IMPORTANT FILE

This is the executable contract every agent validates against. Extract the exact values from [reference/rpd-platform-prototype.html](#) and cross-check against the deck. Write them out in full — no "see prototype" references.

Required content:

Workflow template — 14 steps

| ID    | Name                  | Kind   | Base weeks |
|-------|-----------------------|--------|------------|
| PDD-A | Marketing Brief       | design | 2          |
| PDD-B | Concept Study         | design | 2          |
| PDD-C | Feasibility & Costing | design | 2          |

| ID    | Name                        | Kind   | Base weeks |
|-------|-----------------------------|--------|------------|
| PDD-D | Final Tech Brief & Kick-off | design | 1          |
| PDD-E | Design Detailing            | design | 4          |
| PDD-F | Proof of Concept            | lab    | 3          |
| PDD-G | Design Refinement           | design | 2          |
| PDD-H | Certification               | lab    | 4          |
| PDD-I | Tooling & Sourcing          | design | 3          |
| PDD-J | Pre-Pilot Validation        | lab    | 2          |
| PDD-K | TF-2                        | design | 2          |
| PDD-L | Pilot                       | lab    | 3          |
| PDD-M | Buy-off                     | design | 2          |
| PDD-N | Commercialization           | design | 1          |

### Category multipliers

$A+ = 1.0$ ,  $A = 0.8$ ,  $B = 0.5$ ,  $C = 0.25$   
 $\text{duration\_weeks} = \max(1, \text{round}(\text{base\_weeks} \times \text{multiplier}))$

### Hubs and lab-region mapping

Hubs: R&D-Greece, R&D-India, PD-India, PD-Romania, OEM-HCK, OEM-Seltek  
 Lab region: R&D-Greece→Greece, R&D-India→India, PD-India→India,  
 PD-Romania→Romania, OEM-HCK→India, OEM-Seltek→India

### Horizon constants

$\text{CURRENT\_WEEK} = 31$ ,  $\text{WITHIN\_YEAR\_WEEK} = 52$ ,  $\text{HORIZON\_WEEKS} = 78$

### Scheduling order (serial schedule-generation scheme)

Sort all projects by, in order:

1. **frozen** descending (frozen projects scheduled first, dates locked)
2. Priority:  $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4 \rightarrow Q$

3. Status: In Buyoff (0) → Under Industrialization (1) → In Development (2) → In Queue (3)
4. Category: A+ → A → B → C

## Booking rules

- **Design step:** the project's assigned leader must be free for every week in  $[w, w + \text{duration})$ . Non-frozen projects advance  $w$  until a free window is found.
- **Lab step:** a chamber in the project's lab region whose `allowed_stages` includes this step, with concurrent project count  $< \text{chamber.max}$  for every week in the window. Consumption is  $0.5$  per project-week.
- **Frozen projects:** dates are locked at `actual_start`; capacity is consumed regardless of conflict. A double-booked engineer raises `ENG_CONFLICT`; an over-capacity chamber raises `OVERLAP`.
- **LEFT\_OUT:** no feasible window before `HORIZON_WEEKS`.
- **CAT\_NOT\_ALLOWED:** assigned leader's `allowed_categories` excludes the project's category (or `OEM` for OEM-hub projects). Warning, not a scheduling block.
- **Completing within year:**  $!left\_out \text{ AND } (last\_step\_end + delay) \leq \text{WITHIN\_YEAR\_WEEK}$ . Otherwise `SPILOVER`.
- Status `Commercialized` and `On Hold` are excluded from scheduling entirely.

## Prioritization scoring — 13 dimensions

| Pillar               | Dimension             | Weight |
|----------------------|-----------------------|--------|
| Strategic Alignment  | Strategic Project     | 25     |
| Strategic Alignment  | New Customer          | 25     |
| Strategic Alignment  | New Options           | 25     |
| Regulatory & Quality | Regulatory Compliance | 25     |
| Regulatory & Quality | Quality Improvements  | 25     |
| Financial Return     | RM Savings            | 25     |
| Financial Return     | Total RM Savings      | 25     |
| Financial Return     | Gross Margins         | 25     |
| Financial Return     | Profitability         | 25     |
| Market & Volume      | Annual Volume         | 15     |

| Pillar                   | Dimension                   | Weight |
|--------------------------|-----------------------------|--------|
| Market & Volume          | 3-Year Volume               | 15     |
| Market & Volume          | New Models                  | 15     |
| Investment & Feasibility | CAPEX Investment (inverted) | 10     |

$TOTAL\_WEIGHT = 280$   
 $weighted\_score = \sum(dimension\_score \times weight)$  where each score  $\in [1..5]$   
 $normalised\_pct = round(weighted\_score / (280 \times 5) \times 100)$   
 Bands:  $\geq 70 \rightarrow P1$ ,  $\geq 55 \rightarrow P2$ ,  $\geq 40 \rightarrow P3$ , else  $P4$

Hard gates (override band, force P1)

- Regulatory deadline within 6 months
- Customer certification at risk (Coke/Pepsi)
- Active safety non-compliance

Currency

Base EUR. EUR 1.0, USD 1.08, INR 97 — rates must be configurable at runtime, not constants.

KNOWN DEFECTS IN THE PROTOTYPE — do not silently replicate or silently fix

These three are **open decisions**. Each requires an ADR before implementation. Recording them here so no agent "fixes" them without approval and no agent copies them blindly.

1. **FTE is not applied in scheduling.** The Capacity Planning surface reports capacity scaled by FTE (0.5 FTE  $\rightarrow$  9.4 weeks, 1.0 FTE  $\rightarrow$  18.9 weeks), but the scheduler treats every engineer as fully available for whole weeks. Capacity reporting and the Gantt therefore disagree.  $\rightarrow$  OPEN\_QUESTIONS.md #2.
2. **Chamber efficiency, weeks\_per\_chamber and platforms are display-only.** Only max concurrent gates booking.  $\rightarrow$  OPEN\_QUESTIONS.md #3.
3. **Delay is not propagated.** It is applied once at the end ( $end + delay \leq 52$ ) rather than pushed into downstream steps.  $\rightarrow$  OPEN\_QUESTIONS.md #5.

Invariants (the workflow-auditor asserts all of these on every solver run)

- I1: No engineer is assigned two design steps in the same week — unless at least one project is frozen, in which case ENG\_CONFLICT is raised.
- I2: No chamber exceeds max concurrent projects in any week — unless frozen, then OVERLAP.

- I3: Steps within a project are strictly sequential and non-overlapping; `step[n].start >= step[n-1].end + 1`.
- I4: A lab step is only ever booked to a chamber in the project's lab region whose `allowed_stages` contains that step.
- I5: Every scheduled project has either a complete 14-step schedule or `LEFT_OUT = true`.
- I6: Total design load per hub =  $\Sigma$  design step durations for that hub's projects. Must reconcile with the Capacity surface to the week.
- I7: Total lab load per hub =  $\Sigma$  (lab step durations  $\times$  0.5). Must reconcile with the Capacity surface.
- I8: Re-running the greedy scheduler on identical input produces byte-identical output (determinism).
- I9: `within_year` count on the Dashboard equals the count of projects satisfying the within-year rule in the current schedule run. No independent calculation anywhere.
- I10: Applying priorities never mutates a `frozen` project's dates.

---

## 7. docs/IMPLEMENTATION\_PLAN.md

Phased plan. Every task gets a stable ID (`P2-T04`), an owning agent, explicit dependencies, acceptance criteria, and a status field (`TODO` / `IN_PROGRESS` / `BLOCKED` / `REVIEW` / `DONE`).

Use exactly these phases:

| Phase | Name                                                                       | Owner              | Gate                              |
|-------|----------------------------------------------------------------------------|--------------------|-----------------------------------|
| P0    | Spec lock & open questions                                                 | orchestrator       | client sign-off on OPEN_QUESTIONS |
| P1    | Data model, migrations, seed import                                        | backend-builder    | qa + security                     |
| P2    | Scheduling engine (greedy $\rightarrow$ golden tests $\rightarrow$ CP-SAT) | algorithm-engineer | qa + workflow                     |
| P3    | API layer, auth, RBAC, audit log                                           | backend-builder    | qa + security                     |

| Phase | Name                                                   | Owner                      | Gate          |
|-------|--------------------------------------------------------|----------------------------|---------------|
| P4    | Design system + the six surfaces                       | frontend-builder           | qa + workflow |
| P5    | Scenarios, versioning, history, exports                | backend + frontend         | qa + workflow |
| P6    | Security hardening, SSO, Docker Compose, observability | security-auditor + backend | security      |
| P7    | UAT, data migration, handover docs                     | all                        | all three     |

Populate P0 through P2 with full task detail now. Sketch P3–P7 at one line per task; they get expanded when their phase opens.

**P2 must include, as blocking tasks, in this order:**

- **P2-T01** Implement the greedy SGS scheduler **from docs/DOMAIN\_RULES.md**, not by transliterating the prototype. Read the prototype only to resolve ambiguity in the deck, and record each such resolution in MEMORY.md.
- **P2-T02** Build the differential oracle: run the prototype in headless Chrome against the seed dataset, capture its full schedule output as JSON, commit under **tests/oracle/**. Mark the directory as temporary — it is deleted at P2 close.
- **P2-T03** Run the diff. Produce **docs/ORACLE\_DIVERGENCE.md** listing every difference, each classified as **INTENTIONAL** (with ADR reference) or **PORT\_BUG** (with fix).  
**Acceptance: zero unclassified divergences.** A large number of classified divergences is an acceptable and expected outcome — the prototype has known defects.
- **P2-T04** Implement invariant assertions I1–I10 as a reusable validator. These, not the oracle, are the permanent correctness contract.
- **P2-T05** Replace the oracle fixtures with spec-derived test cases: hand-constructed scenarios that exercise each rule in DOMAIN\_RULES.md in isolation (single engineer contention, chamber saturation, frozen conflict, left-out at horizon, category mismatch, spillover boundary at week 52). Delete **tests/oracle/**.
- **P2-T06** CP-SAT model: intervals + **AddNoOverlap** per engineer, **AddCumulative** per chamber, objective = maximise weighted value completing by week 52
- **P2-T07** Solver comparison harness: greedy vs CP-SAT on the same input, diff report
- **P2-T08** Monte Carlo delivery forecast (P50/P80) on top of the solver

---

## 8. docs/MEMORY.md

Create it with a header explaining the append-only rule and this entry template:

### [YYYY-MM-DD HH:MM] <TASK-ID> — <agent-name>

\*\*Did:\*\* one or two lines

\*\*Files touched:\*\* paths

\*\*Decisions:\*\* any choice made that a future agent could plausibly make differently

\*\*Deviations from plan:\*\* none | description + why

\*\*Broke / discovered:\*\* anything surprising, any invariant that failed

\*\*Gate result:\*\* N/A | PASS | FAIL (+ reason)

\*\*Next:\*\* what should happen next

---

Seed it with one entry recording this bootstrap session.

Also add a **## Standing Decisions** section at the top, above the log, holding the short list of choices that override defaults (e.g. "custom Gantt, no commercial library"). Agents read this section first.

---

## 9. docs/OPEN\_QUESTIONS.md

Numbered, each with: the question, why it matters, what we're assuming until answered, and who must answer it. Seed with these eight:

1. Are the 14 steps strictly sequential, or is there a real precedence DAG with parallel branches?
2. Should FTE gate throughput? (0.5 FTE = half-speed, or half the available weeks?)
3. Do chamber **efficiency** and **weeks\_per\_chamber** constrain booking, or are they reporting-only?
4. Working calendars per hub — Greek, Indian and Romanian holidays differ substantially.

5. Should delay propagate into downstream steps, or stay a terminal adjustment?
  6. What is the real horizon — the Gantt shows 2026–2027 but the logic caps at 78 weeks.
  7. Excel import/export contract — is the existing spreadsheet the system of record during transition?
  8. **GDPR**: per-engineer utilization is personal data on EU employees (Greece, Romania). Does Frigoglass's DPO approve, and is works council consultation required? Blocking for P4.
  9. Which IdP, and can we get an app registration?
- 

## 10. Subagents — `.claude/agents/*.md`

Each file uses this frontmatter:

---

name: <kebab-name>

description: <when the orchestrator should delegate to this agent>

tools: Read, Write, Edit, Bash, Glob, Grep

model: inherit

---

Every agent's system prompt **must** open with the Memory Protocol restated, and **must** close with "append your MEMORY.md entry before reporting back."

### rpd-orchestrator

Supervises. Reads all four docs, decides the next task from IMPLEMENTATION\_PLAN.md, delegates to exactly one specialist at a time, reviews returned work against DOMAIN\_RULES.md, runs gate checks, updates plan status, and refuses to advance a phase without three PASSES. **Writes no application code**. If a specialist returns work that violates an invariant or the locked stack, the orchestrator rejects it and issues a remediation task rather than patching it. Owns `docs/MEMORY.md` structure and `docs/ADR/`.

### backend-builder

FastAPI, SQLAlchemy 2.0 async models, Alembic migrations, Pydantic v2 schemas, Celery tasks, Redis integration, MinIO, OIDC auth, RBAC + hub-scoped row filtering, the append-only

audit log, SSE endpoints. Owns `backend/` excluding `backend/scheduling/`. Rules: every endpoint gets a Pydantic request and response model; every mutation writes an audit row; every query that touches project data goes through the hub-scoping layer; no raw SQL string interpolation; financial columns use the encrypted type.

## algorithm-engineer

Owns `backend/scheduling/` and nothing else. Implements the greedy SGS port, the golden-file harness, the invariant validator I1–I10, the CP-SAT model, the comparison harness, the scoring engine and the Monte Carlo forecaster. Rules: pure functions — the scheduler takes a dataclass input and returns a dataclass output, with no DB or network access. Determinism is mandatory (fixed seeds, sorted iteration, no set iteration order dependence). Implements from `DOMAIN_RULES.md`, never by transliterating the prototype; the prototype is a differential oracle during P2 and is retired at P2 close. Any intentional divergence from prototype behaviour requires an ADR and an entry in `docs/ORACLE_DIVERGENCE.md`. CP-SAT never runs in a request-handling process. Uses the `scheduling-algorithms` skill.

## frontend-builder

Owns `frontend/`. React 19 + TS + Vite, the design system, all six surfaces, the custom Gantt. Uses the `ui-ux-pro-max`, `frontend-builder` and `dataviz-gantt` skills. Rules: no business logic in the frontend — the schedule comes from the API, the frontend only renders it. Every number displayed traces to a single API field (invariant I9). Virtualize any list over 100 rows. `prefers-reduced-motion` respected everywhere. No `dangerouslySetInnerHTML`. Bundle budget: 400 KB gzipped initial, route-split beyond that. `three.js` is forbidden in v1.

## qa-inspector

Writes and runs tests; does not write feature code. `pytest` + `pytest-asyncio` + `testcontainers` for the backend, `Vitest` + `Testing Library` for units, `Playwright` for the six surfaces. Enforces:  $\geq 85\%$  coverage on `backend/scheduling/`,  $\geq 70\%$  elsewhere; contract tests that the OpenAPI schema matches the TS types; golden-file tests green; accessibility pass (axe) on every surface. Reports PASS/FAIL per phase gate into `MEMORY.md` with specifics, never a bare verdict. Uses the `qa-testing` skill.

## workflow-auditor

The domain referee. Does not write feature code. On every solver-touching change it: re-derives the schedule independently from `DOMAIN_RULES.md`, asserts I1–I10, reconciles the Dashboard/Capacity/Gantt numbers against a single source, and diffs greedy vs CP-SAT for unexplained divergence. During P2 it also verifies that every entry in `docs/ORACLE_DIVERGENCE.md` is classified and that no agent has silently changed behaviour on one of the three known prototype defects without an ADR. Has authority to FAIL a phase gate on its own.

## security-auditor

OWASP ASVS L2 review of every phase. Checks: authn/authz on every endpoint including negative tests, hub-scoping cannot be bypassed by parameter tampering, secrets absent from repo and logs, dependency scan (`pip-audit`, `npm audit`, Trivy on images), CSP and security headers, CSRF on mutations, financial-field encryption at rest, no PII in logs, audit log immutability, TLS config, GDPR items from OPEN\_QUESTIONS #8. Produces a findings table with severity; any High or Critical FAILs the gate. Uses the `security-review` skill.

---

## 11. Skills — `.claude/skills/*/SKILL.md`

Frontmatter: `name`, `description`. Write real, opinionated content in each — not placeholders.

### ui-ux-pro-max

The visual bar for an industrial B2B dashboard that executives will present to a board. Cover: information density done right (this is a data tool, not a marketing page); a type scale with tabular numerals for every figure; a restrained palette anchored on Frigoglass blue with semantic colour reserved for status only; 8px spacing grid; card and panel patterns; table design (sticky headers, zebra-free, right-aligned numerics, inline sparklines); empty, loading and error states for every surface; skeleton loaders over spinners; how to show conflict/warning states without a wall of red. Explicit anti-patterns: gradient hero backgrounds, decorative accent stripes, glassmorphism, unlabelled icons, colour as the sole carrier of meaning, animation on data updates that fires more than once per interaction.

### frontend-builder

React 19 + TS + Vite conventions. Folder structure, TanStack Query key factory and cache invalidation strategy, Zustand store slicing for scenario edits with undo/redo, form handling with react-hook-form + zod, shadcn/ui setup and when to pull from 21st.dev instead of writing new, Framer Motion patterns (`layoutId` for the Gantt row expansion, `AnimatePresence` for panels, spring configs — never linear easing on layout), Lottie usage boundaries (empty state, solver progress, save confirmation; nothing else), code splitting per route, and the bundle budget enforcement.

### dataviz-gantt

How to build the custom Gantt. Week-bucket coordinate system, virtualization over 236 projects × 14 steps, the hatched-bar SVG pattern for actual+delay, the red delay connector, badge placement without overlap at narrow zoom, the activity-load bottleneck strip, zoom levels (weeks/days), horizontal + vertical scroll sync between the fixed left column and the timeline,

and canvas fallback if SVG node count exceeds ~10k. Plus ECharts config for the utilization heatmap and Recharts config for the dashboard panels.

## scheduling-algorithms

CP-SAT modelling patterns for RCPSP: `NewIntervalVar`, `AddNoOverlap` for unary resources (engineers), `AddCumulative` for capacitated resources (chambers), precedence via `Add(start[i+1] >= end[i])`, optional intervals with `NewOptionalIntervalVar` for the left-out/deferred decision, objective construction for maximising weighted within-year completion, solution hinting from the greedy result to warm-start, `max_time_in_seconds` and `num_search_workers` tuning, and how to extract and explain infeasibility. Also: the greedy SGS reference implementation, determinism requirements, and how to write the golden-file harness.

## security-review

OWASP ASVS L2 checklist mapped onto this stack, the RBAC/hub-scoping test matrix, secret scanning, dependency and container scanning commands, the security header set, CSP for a Vite SPA, the audit-log immutability test, encrypted-column verification, and the GDPR checklist for employee utilization data.

## qa-testing

pytest/testcontainers Postgres fixtures, factory-boy patterns for the domain objects, the golden-file comparison helper, Vitest + Testing Library conventions, Playwright page objects for the six surfaces, axe integration, coverage thresholds, and how to write a gate report.

---

## 12. Slash commands — `.claude/commands/`

- `start-task.md` — takes a task ID. Reads DOMAIN\_RULES → MEMORY → IMPLEMENTATION\_PLAN, confirms dependencies are DONE, sets status IN\_PROGRESS, delegates to the owning agent.
- `close-task.md` — takes a task ID. Verifies acceptance criteria, requires a MEMORY.md entry, runs the relevant test suite, sets status DONE or REVIEW.
- `gate-check.md` — takes a phase ID. Invokes qa-inspector, workflow-auditor and security-auditor in sequence, collects three verdicts, writes a combined gate entry to MEMORY.md, and advances the phase only on three PASSES.

## 13. `.claude/settings.json`

Permissions allowing read/write within the repo, `Bash` for the standard toolchain (`uv/pip`, `pytest`, `npm`, `docker compose`, `alembic`, `ruff`, `mypy`, `eslint`, `tsc`). Deny writes to

[reference/](#). Add a hook that blocks any commit containing a string matching common secret patterns.

---

## 14. When you are finished

1. Print the created file tree.
2. Print [docs/OPEN\\_QUESTIONS.md](#) in full — these go to the client before P1 starts.
3. Print the P0 and P1 task lists from [docs/IMPLEMENTATION\\_PLAN.md](#).
4. State explicitly which of the three known prototype defects need ADRs before P2 can begin.
5. Stop. Do not start P1.